arXiv:2403.19647v3 [cs.LG] 27 Mar 2025

SPARSE FEATURE CIRCUITS: DISCOVERING AND EDITING INTERPRETABLE CAUSAL GRAPHS IN LANGUAGE MODELS

Samuel Marks*
Northeastern University

Can Rager
Independent

Eric J. Michaud
MIT

Yonatan Belinkov
Technion – IIT

David Bau
Northeastern University

Aaron Mueller*
Northeastern University

ABSTRACT

We introduce methods for discovering and applying **sparse feature circuits**. These are causally implicated subnetworks of human-interpretable features for explaining language model behaviors. Circuits identified in prior work consist of polysemantic and difficult-to-interpret units like attention heads or neurons, rendering them unsuitable for many downstream applications. In contrast, sparse feature circuits enable detailed understanding of unanticipated mechanisms in neural networks. Because they are based on fine-grained units, sparse feature circuits are useful for downstream tasks: We introduce SHIFT, where we improve the generalization of a classifier by ablating features that a human judges to be task-irrelevant. Finally, we demonstrate an entirely unsupervised and scalable interpretability pipeline by discovering thousands of sparse feature circuits for automatically discovered model behaviors.

1 INTRODUCTION

The key challenge of interpretability research is to scalably explain the many unanticipated behaviors of neural networks (NNs). Much recent work explains NN behaviors in terms of coarse-grained model components, for example by implicating certain induction heads in in-context learning (Olsson et al., 2022) or MLP modules in factual recall (Meng et al., 2022; Geva et al., 2023; Nanda et al., 2023, *inter alia*). However, such components are generally polysemantic (Elhage et al., 2022) and hard to interpret, making it difficult to apply mechanistic insights to downstream applications. On the other hand, prior methods for analyzing behaviors in terms of fine-grained units (Kim et al., 2018; Belinkov, 2022; Geiger et al., 2023; Zou et al., 2023) attempt to fit model internals to researcher-specified mechanistic hypotheses using researcher-curated data. These approaches are not well-suited to the many cases where researchers cannot anticipate ahead of time how models internally implement their surprising behaviors.

We propose to explain model behaviors using fine-grained components that play narrow, interpretable roles. Doing so requires us to address two challenges: First, we must identify an appropriate fine-grained unit of analysis, since obvious choices like neurons[1] are rarely interpretable, and units discovered via supervised methods like linear probing require pre-existing hypotheses (Mueller et al., 2024). Second, we must address the scalability problem posed by searching for causal circuits over a large number of fine-grained units.

We leverage recent progress in dictionary learning for NN interpretability (Bricken et al., 2023; Cunningham et al., 2024) to tackle the first challenge. Namely, we use **sparse autoencoders** (SAEs) to identify directions in LM latent spaces which represent human-interpretable concepts. Then, to address the scalability challenge, we employ linear approximations (Sundararajan et al., 2017; Nanda,

*Correspondence to s.marks@northeastern.edu and aa.mueller@northeastern.edu.

[1]We use "neuron" to refer to a basis-aligned direction in an LM's latent space (not necessarily preceded by a nonlinearity).

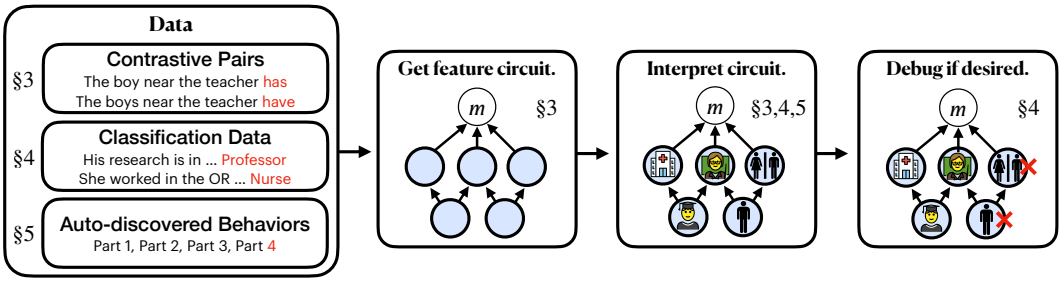

Figure 1: Overview. Given contrastive input pairs, classification data, or automatically discovered model behaviors, we discover circuits composed of human-interpretable sparse features to explain their underlying mechanisms. We then label each feature according to what it activates on or causes the model to predict. Finally, if desired, we can ablate spurious features out of the circuit to modify how the system generalizes.

2022; Syed et al., 2023) to efficiently identify SAE features which are most causally implicated in model behaviors, as well as connections between these features. The result is a **sparse feature circuit** which explains how model behaviors arise via interactions among fine-grained human-interpretable units.

Sparse feature circuits can be productively used in downstream applications. We introduce a technique, Sparse Human-Interpretable Feature Trimming (SHIFT; §4), which shifts the generalization of an LM classifier by surgically removing sensitivity to unintended signals. Unlike previous work on spurious cue removal—which isolates spurious signals using disambiguating data—SHIFT identifies unintended signals using interpretability and human judgement. We thus showcase SHIFT by debiasing a classifier in a worst-case setting, where an unintended signal (gender) is *perfectly predictive* of target labels (profession).

Finally, we demonstrate our method's scalability by automatically discovering thousands of narrow LM behaviors—for example, predicting "to" as an infinitive object or predicting commas in dates—with the clustering approach of Michaud et al. (2023), and then automatically discovering feature circuits for these behaviors (§5).

Our contributions are summarized as follows (Figure 1):

1. A scalable method to discover sparse **feature circuits**. We validate our method by discovering and evaluating feature circuits on a suite of subject-verb agreement tasks.
2. **SHIFT**, a technique for removing a LM classifier's sensitivity to unintended signals, even without data that isolate these signals.
3. A *fully-unsupervised* pipeline for computing feature circuits for thousands of automatically discovered LM behaviors, viewable at `feature-circuits.xyz`.

We release code, data and autoencoders at `github.com/saprmarks/feature-circuits`.

2 FORMULATION

Feature disentanglement with sparse autoencoders. A fundamental challenge in NN interpretability is that individual neurons are rarely interpretable (Elhage et al., 2022). Therefore, many interpretability researchers have recently turned to **sparse autoencoders** (SAEs), an unsupervised technique for identifying a large number of interpretable NN latents (Cunningham et al., 2024; Bricken et al., 2023; Templeton et al., 2024; Rajamanoharan et al., 2024a;b). Given a model component with latent space $\mathbb{R}^d$ and an activation $\mathbf{x} \in \mathbb{R}^d$, an SAE computes a decomposition

$$\mathbf{x} = \hat{\mathbf{x}} + \boldsymbol{\epsilon}(\mathbf{x}) = \sum_{i=1}^{d_{\text{SAE}}} f_i(\mathbf{x})\mathbf{v}_i + \mathbf{b} + \boldsymbol{\epsilon}(\mathbf{x}) \tag{1}$$

into an approximate reconstruction $\hat{\mathbf{x}}$ as a sparse sum of features $\mathbf{v}_i$ and an *SAE error term* $\boldsymbol{\epsilon}(\mathbf{x}) \in \mathbb{R}^d$. Here d_{SAE} is the *width* of the SAE, the *features* $\mathbf{v}_i \in \mathbb{R}^d$ are unit vectors, the *feature activations* $f_i(\mathbf{x}) \geq 0$ are a sparse set of coefficients, and $\mathbf{b} \in \mathbb{R}^d$ is a bias. SAEs are trained on an objective

which promotes having a small reconstruction error $\|\mathbf{x}-\hat{\mathbf{x}}\|_2$ while using only a sparse set of feature activations $f_i(\mathbf{x})$. Rather than discard the error terms $\boldsymbol{\epsilon}$ for the purposes of circuit discovery, our methods handle them gracefully by incorporating them into our sparse feature circuits; this gives a principled decomposition of model behaviors into contributions from interpretable features and error components not yet captured by our SAEs.

In this work, we leverage the following suites of SAEs:

- A suite of SAEs we train for each sublayer (attention layer, MLP, residual stream, and embeddings) of Pythia-70M (Biderman et al., 2023). We closely follow Bricken et al. (2023), using a ReLU-linear encoder f_i and sparse dimension $d_{\text{SAE}} = 64 \times d$ and training the SAEs to minimize a combination of an L2 reconstrution loss and L1 regularization term which promotes sparsity. Details about our Pythia SAEs and their training can be found in Appendix B.1.
- The open source Gemma Scope SAEs (Lieberum et al., 2024) available for all sublayers (excluding embeddings) of the open-weights Gemma-2-2B model (Team et al., 2024). These SAEs use a Jump-ReLU-linear encoder and $d_{\text{SAE}} = 8 \times d$. Details about the Gemma Scope SAEs can be found in Appendix B.2.

Scalably training better SAEs is an active area of research, as illustrated by the ready availability of open-source SAEs (Gao et al., 2024; Lieberum et al., 2024; Lin & Bloom, 2023). Thus, our goal is to—given a suite of trained SAEs—scalably apply them to understand NN behaviors; we treat scaling the SAEs themselves as out-of-scope.

Attributing causal effects with linear approximations. Let m be a real-valued metric computed via a computational graph (e.g., a NN); let $\mathbf{a}$ represent a node in this graph. Following prior work (Vig et al., 2020; Finlayson et al., 2021), we quantify the importance of $\mathbf{a}$ on a pair of inputs $(x_{\text{clean}}, x_{\text{patch}})$ via its *indirect effect* (IE; Pearl, 2001) on m:

$$\text{IE}(m; \mathbf{a}; x_{\text{clean}}, x_{\text{patch}}) = m\left(x_{\text{clean}} | \text{do}(\mathbf{a} = \mathbf{a}_{\text{patch}})\right) - m(x_{\text{clean}}). \tag{2}$$

Here, $\mathbf{a}_{\text{patch}}$ is the value that $\mathbf{a}$ takes in the computation of $m(x_{\text{patch}})$, and $m(x_{\text{clean}}|\text{do}(\mathbf{a} = \mathbf{a}_{\text{patch}}))$ denotes the value of m when computing $m(x_{\text{clean}})$ but *intervening* in the computation of m by manually setting $\mathbf{a}$ to $\mathbf{a}_{\text{patch}}$. For example, given inputs x_{clean} ="The **teacher**" and x_{patch} ="The **teachers**," we have metric $m(x) = \log P(\text{"are"}|x) - \log P(\text{"is"}|x)$, the log probability difference output by the LM. Then if $\mathbf{a}$ is the activation of a particular neuron, a large value of $\text{IE}(m; \mathbf{a}; x_{\text{clean}}, x_{\text{patch}})$ indicates that the neuron is highly influential on the model's decision to output "is" vs. "are" on this pair of inputs.

We often want to compute IEs for a very large number of model components $\mathbf{a} \in \mathbb{R}^d$, which cannot be done efficiently with (2). We thus employ linear approximations to (2) that can be computed for many $\mathbf{a}$ in parallel. The simplest such approximation, *attribution patching* (Nanda, 2022; Syed et al., 2023; Kramár et al., 2024), employs a first-order Taylor expansion

$$\hat{\text{IE}}_{\text{atp}}(m; \mathbf{a}; x_{\text{clean}}, x_{\text{patch}}) = \nabla_{\mathbf{a}} m\big|_{\mathbf{a}=\mathbf{a}_{\text{clean}}} (\mathbf{a}_{\text{patch}} - \mathbf{a}_{\text{clean}}) \tag{3}$$

which estimates (2) for every $\mathbf{a}$ in parallel using only two forward and one backward pass.

To improve the quality of the approximation, we can instead employ a more expensive but more accurate approximation based on integrated gradients (Sundararajan et al., 2017; Hanna et al., 2024):

$$\hat{\text{IE}}_{\text{ig}}(m; \mathbf{a}; x_{\text{clean}}, x_{\text{patch}}) = \frac{1}{N} \left(\sum_{\alpha} \nabla_{\mathbf{a}} m\big|_{\alpha \mathbf{a}_{\text{clean}} + (1-\alpha)\mathbf{a}_{\text{patch}}} \right) (\mathbf{a}_{\text{patch}} - \mathbf{a}_{\text{clean}}) \tag{4}$$

where the sum in (4) ranges over $N = 10$ equally-spaced $\alpha \in \{0, \frac{1}{N}, \ldots, \frac{N-1}{N}\}$. This cannot be done in parallel for two nodes when one is downstream of another, but can be done in parallel for arbitrarily many nodes which do not depend on each other. Thus the additional cost of computing $\hat{\text{IE}}_{\text{ig}}$ over $\hat{\text{IE}}_{\text{atp}}$ scales linearly in N and the serial depth of m's computation graph.

The above discussion applies to the setting where we have a pair of clean and patch inputs, and we would like to understand that effect of patching a particular node from its clean to patch values. But in some settings (e.g., §4, 5), we have only a single input x. In this case, we instead use a *zero-ablation*, using the indirect effect $\text{IE}(m; \mathbf{a}; x) = m(x|\text{do}(\mathbf{a} = \mathbf{0})) - m(x)$ from setting $\mathbf{a}$ to $\mathbf{0}$. We get the modified formulas for $\hat{\text{IE}}(m; \mathbf{a}; x)$ from (3) and (4) by replacing $\mathbf{a}$ with $\mathbf{0}$.